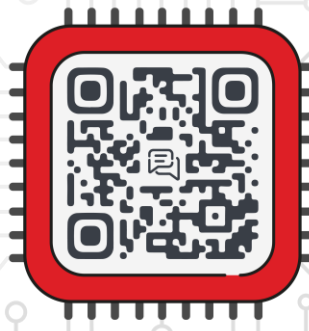




7

16

2400 sp

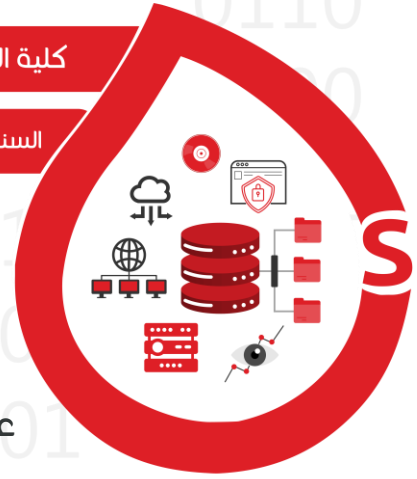
عملي

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

TRANSACTION

عملي مشترك



RB Informatcs; 8/12/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

تواجه قواعد البيانات (DB) مشكلتين مهمتين من أكبر المشاكل القاتلة فيها، وهما (Dead Lock & Lock)
وسبب الـ Lock والـ Dead Lock هو المعطيات (Data).
في حال قامت أي Session بحفظ التعديلات، أي تم تخزين بيانات في Table وما زال (Draft) ولم يتم الحفظ بعد،
يتم منع الكتابة في Table.
القراءة من Table مختلفة عن القراءة من Draft، حيث ستقرأ السجلات المخزنة ولن تقرأ من Draft.

Transaction

نُستخدم الـ Transaction عندما نريد تنفيذ عدة عمليات DML على الـ DB دفعة واحدة.

مثلاً: عند تنفيذ عملية تحويل رصيد، يوجد عدة استعلامات يتم تنفيذها على الـ DB، وفي حال فشل أي استعلام قد يسبب مشكلة.
فلو تم تحويل الرصيد إلى حساب آخر دون طرح المبلغ من الحساب الذي قام بعملية التحويل، ستصبح البيانات المخزنة خاطئة.

لذلك نستخدم Transaction لتخزين عدة تعليمات DML، إما أن تنتهي بـ:

Commit: ويحدث عندما يتم تنفيذ جميع التعليمات.

Rollback: ويتم التراجع عن جميع العمليات التي تم تنفيذها في حال فشل إحداها.

إذاً، الـ Transaction يعمل بمبدأ الكل أو لا شيء، في حال تم تنفيذ التعليمات كاملة يتم تأكيد عملية التنفيذ (Commit)،
وفي حال فشل واحدة على الأقل يتم التراجع عن الكل (Rollback) ويكون وكأنه لم يُنفذ أي تعليمة.



الحرية ليست فرصة للقيام بما يحلو لك، بل

هي فرصة لتكون أفضل.

❖ اذا ماهو مبدأ عمل Transaction؟

تعمل الـ Transaction وفق مبدأ:

الكل أو لا شيء (All or Nothing)

- إذا تم تنفيذ جميع التعليمات بنجاح يتم تثبيت التعديلات باستخدام COMMIT.
 - إذا فشلت أي تعليمة أثناء التنفيذ يتم التراجع عن جميع التعديلات باستخدام ROLLBACK.
- وبالتالي، لا يُسمح ببقاء البيانات في حالة غير متكاملة.

أي أن الـ Transaction تُعامل مجموعة الأوامر على أنها عملية واحدة غير قابلة للتجزئة.

يدعم SQL الحفظ التلقائي (Auto Commit)، أي بعد تنفيذ أي تعليمة على الـ DB يتم عمل Commit لها تلقائياً.

في الـ Transaction لا يتم عمل Commit بعد كل تعليمة على حدة، وإنما بعد تنفيذ كامل التعليمات المذكورة في الـ Transaction يتم عمل Commit لها كـ Block واحدة.

■ العلاقة بين Transaction و Auto Commit:

- في الوضع الطبيعي يدعم SQL خاصية Auto Commit، حيث يتم حفظ كل تعليمة مباشرة بعد تنفيذها.
- عند استخدام Transaction يتم إيقاف الحفظ التلقائي مؤقتاً.
- لا يتم حفظ التعديلات إلا بعد تنفيذ أمر COMMIT.
- في حال عدم تنفيذ COMMIT يمكن التراجع عن التعديلات باستخدام ROLLBACK.

■ مثال للتوضيح

T1		
A(PK)	B	C
1	×W	¥Z

T2	
A(FK)	B
1	×W

T3	
A(FK)	C
1	¥Z

```
BEGIN TRANSACTION
```

```
Update T1
```

```
Set B = 'w',
```

```
C = 'z'
```

```
Where A = 1;
```

```

Update T2
Set B = 'w'

Where A = 1;

Update T3
Set C = 'z'

Where A = 1;

COMMIT TRANSACTION

```

في المثال الموضح:

- نريد تنفيذ ثلاث تعليمات Update على الجداول 1 و 2 و 3، والجداول مترابطة عن طريق FK.
 - حين تتغير قيمة ما في الجدول 1 نريدها أن تتغير في الجداول 2 و 3 لتحمل نفس القيمة، وبذلك نضمن الحصول على Data متكاملة (Integrity).
 - في حال لم نستخدم Transaction وقمنا بتنفيذ كل تعليمة Update على حدة، قد يتم تعديل قيمة في t1 دون تعديلها في t2، وبذلك لا تكون البيانات متكاملة.
- مثال: تم تعديل علامتك في جدول "علامات الخوارزميات" دون تعديلها في جدول "علامات الطالب" لتحمل نفس القيمة، عندها نقول إن البيانات غير متكاملة.

لنعم حدوث مثل هذه الأخطاء نستخدم Transaction ليتم تنفيذ عدة تعليمات كـ Block واحدة، إما أن تنتهي بـ Commit أو Rollback.

■ مثال 2 للتوضيح:

في حال تنفيذ عملية تحويل رصيد بين حسابين:

- يتم طرح المبلغ من الحساب الأول.
 - يتم إضافة المبلغ إلى الحساب الثاني.
- إذا تم تنفيذ عملية الإضافة دون عملية طرح، تصبح البيانات خاطئة.
- لذلك يتم وضع العمليتين ضمن Transaction واحدة لضمان تنفيذهما معاً أو إلغائهما معاً.

Lock Table

- أثناء تنفيذ Transaction فعالة، يتم حجز (Lock) على الجداول أو الصفوف التي يتم التعديل عليها، وذلك لمنع أي Session أخرى من التعديل على نفس البيانات في نفس الوقت.
- في حال محاولة Session أخرى تنفيذ استعلام على جدول مقفول:
- قد يبقى الاستعلام في حالة Waiting.
 - يستمر القفل حتى يتم تنفيذ COMMIT أو ROLLBACK.

▪ تأثير Lock على أداء النظام:

- في حال وجود Transaction كبيرة تقوم بقفل عدة جداول أساسية:
- ينخفض أداء قاعدة البيانات.
 - يزداد وقت انتظار الاستعلامات.
 - قد يصل النظام إلى حالة بطء شديد.

▪ طرق معالجة مشكلة Lock:

1. إعادة تصميم قاعدة البيانات:

- حل غير مفضل.
- يُستخدم كحل أخير.

2. Transaction LL Refactoring:

- إعادة كتابة الاستعلامات.
- تحسين الأداء دون تغيير بنية الجداول.

3. استخدام Index أفضل الحلول:

- تقليل مستوى القفل من Table إلى Page أو Row.
- تحسين سرعة الوصول إلى البيانات.

▪ دور Index في تقليل Lock:

- Table Scan يؤدي إلى Lock على كامل الجدول.
- Index Seek يؤدي إلى Lock على الصف المطلوب فقط.

لذلك يُعتبر بناء Index مناسب من أبسط وأفضل الحلول لتقليل مشاكل Lock وتحسين الأداء.

- مثال للشرح: لنفرض وجود جدول باسم emp يحتوي على بيانات الموظفين.

Session 1:

```
BEGIN TRANSACTION;
```

```
UPDATE emp
```

```
SET salary = salary + 100
```

```
WHERE emp_id = 1;
```

في هذه الحالة:

- يتم تنفيذ Transaction.
- يتم عمل Lock على الصف الذي يحمل emp_id = 1.
- يبقى القفل موجوداً طالما لم يتم تنفيذ COMMIT أو ROLLBACK.

Session 2 في نفس الوقت

```
SELECT*
```

```
FROM emp
```

```
WHERE emp_id = 1;
```

- يبقى الاستعلام في حالة Waiting.
 - لا يتم جلب البيانات حتى يتم تحرير القفل.
 - يتم تحرير القفل عند تنفيذ: (COMMIT; or ROLLBACK;)
- التفسير: أثناء تنفيذ Transaction تقوم قاعدة البيانات بحجز البيانات المتأثرة لمنع التعديل أو القراءة غير الصحيحة من قبل Sessions أخرى، مما يضمن سلامة البيانات ومنع التعارض.
- سؤال: كيف يساعدنا الفهرس (Index) في حل مشكلة Lock؟

يساعد استخدام الفهرس (Index) في تقليل مشكلة الـ Lock من خلال تحسين طريقة الوصول إلى البيانات داخل الجداول.

فعند عدم وجود فهرس مناسب، تقوم قاعدة البيانات بتنفيذ الاستعلام باستخدام Table Scan أو Clustered Scan، أي يتم مسح كامل الجدول بحثاً عن البيانات المطلوبة، مما يؤدي إلى فرض Lock على كامل الجدول (Table Lock) طوال مدة تنفيذ العملية.

أما عند وجود فهرس مناسب على الأعمدة المستخدمة في شروط الاستعلام (WHERE)، فإن قاعدة البيانات تنفذ الاستعلام باستخدام Index Seek، حيث يتم الوصول مباشرة إلى الصف المطلوب دون الحاجة إلى مسح كامل الجدول، وبالتالي يتم فرض Lock على الصف (Row Lock) أو الصفحة (Page Lock) فقط بدلاً من قفل الجدول بالكامل.

وبذلك يؤدي استخدام الفهرس إلى:

- تقليل مدة القفل.
- تقليل عدد الصفوف المتأثرة بالقفل.
- تحسين أداء قاعدة البيانات.
- تقليل احتمالية حدوث Lock Table و Dead Lock.

ولهذا يُعد بناء الفهارس المناسبة من أبسط وأفضل الحلول للحد من مشاكل الـ Lock وتحسين كفاءة تنفيذ الـ Transactions.

- مثال توضيحي على دور الفهرس وتأثيره على Execution Plan
لدينا جدول باسم emp يحتوي على بيانات الموظفين، وبنيت كالتالي:

```
Emp)
emp_id    NUMBER    (PK)
emp_name  VARCHAR
dept_id   NUMBER
salary   NUMBER
)
```

- emp_id: رقم الموظف (مفتاح أساسي).
- emp_name: اسم الموظف.
- dept_id: رقم القسم الذي يعمل فيه الموظف.
- salary: راتب الموظف.

emp_id	emp_name	dept_id	Salary
1	Ali	10	3000
2	Sara	20	3500
3	Omar	10	4000
4	Lina	30	3200
5	Noor	10	2800

لاستعلام المستخدم:

```
UPDATE emp
SET salary = salary + 100
WHERE dept_id = 10;
```

■ ماذا يعني ذلك؟

- سيتم تعديل رواتب الموظفين الذين يعملون في القسم رقم 10.
- في مثالنا هؤلاء الموظفون هم:

- Ali
- Omar
- Noor

■ بدون Index على dept_id:

- قاعدة البيانات لا تعرف مكان الصفوف المطلوبة.
- يتم تنفيذ الاستعلام باستخدام Table Scan.
- يتم مسح جميع صفوف جدول emp.
- يتم فرض Lock على كامل الجدول.
- أي Session أخرى تحاول الوصول للجدول ستدخل حالة Waiting .
- مع Index على dept_id:

```
CREATE INDEX idx_emp_dept
ON emp(dept_id);
```

- يتم تنفيذ الاستعلام باستخدام Index Seek.
- يتم الوصول مباشرة إلى الصفوف التي dept_id = 10.
- يتم قفل الصفوف الثلاثة فقط.
- يتم تقليل زمن القفل وتحسين الأداء.

الربط مع Deadlock

■ عند قفل كامل الجدول:

- تزيد فرصة تداخل Transactions.
- يزداد احتمال حدوث Deadlock.

■ أما عند استخدام Index:

- يتم تقليل نطاق القفل.
- تقل فرصة حدوث Deadlock.
- يصبح النظام أكثر استقراراً.

لنقم الان بالتطبيق العملي بناء على الجداول السابقة لدينا :

```
CREATE TABLE dbo.dept (  
    depid INT IDENTITY(1,1) PRIMARY KEY,  
    dname VARCHAR(50) NOT NULL  
);  
  
INSERT INTO dbo.dept VALUES ('admin');  
INSERT INTO dbo.dept VALUES ('sales');  
INSERT INTO dbo.dept VALUES ('archives');  
  
CREATE TABLE dbo.emp(  
    empid INT IDENTITY(1,1) PRIMARY KEY,  
    ename VARCHAR(50) NOT NULL,  
    gender CHAR(1) NOT NULL,  
    salary INT NOT NULL,  
    depid INT FOREIGN KEY REFERENCES dept (depid),  
    CONSTRAINT CHK_emp_salary CHECK (salary > 0)  
);  
  
INSERT INTO dbo.emp (ename, gender, salary, depid) VALUES ('ahmad', 'M', 9400, 1);  
INSERT INTO dbo.emp (ename, gender, salary, depid) VALUES ('sara', 'F', 5800, 2);  
INSERT INTO dbo.emp (ename, gender, salary, depid) VALUES ('maheer', 'm', 80000, 1);  
INSERT INTO dbo.emp (ename, gender, salary, depid) VALUES ('maha', 'F', 8500, 1);  
INSERT INTO dbo.emp (ename, gender, salary, depid) VALUES ('fares', 'M', 7000, 2);
```

لنقم الان بتنفيذ تعليمة ال Begin transaction ونلاحظ انه خلال عملية الإضافة لو قمنا بتنفيذها وحدها سيتم التنفيذ مباشرة ولكن اذا قمنا بعمل ال begin transaction تمسك التعليمة أي انها تبقى قيد التنفيذ حتى نتبعها بال commit او ال RollBack .

```

Select * from dbo.emp
Order by depid
BEGIN TRANSACTION
INSERT INTO dbo.emp VALUES ('xxxx', 'M', 8000, 1);

```

الان لو حاولنا القيام بعملية تنفيذ استعلام اخر في نفس ال session يتم تنفيذه والسبب بانه ال session هي من تمسك ال جدول لذا يمكننا اجراء الاستعلامات عليه ونلاحظ انه ال result set تحتوي على القيم التي قمنا باضافتها على الرغم من انه لم يتم عمل الحفظ لها بعد (commit).

Select * from dbo.emp
Order by depid

134 %

	empid	ename	gender	salary	depid
1	8	ahmad	M	9400	1
2	10	maheer	m	80000	1
3	11	maha	f	8500	1
4	13	xxxx	M	8000	1
5	12	fares	M	7000	2
6	9	sara	F	5800	2

سؤال بالاستعلام Select * from deo.emp في حال لو قمنا بفتح session اخرى ثانية واعدنا تنفيذ نفس الاستعلام هل سنحصل على نفس النتيجة ؟

Select * from dbo.emp
Order by depid

163 %

	empid	ename	gender	salary	depid
1	8	ahmad	M	9400	1
2	10	maheer	m	80000	1
3	11	maha	f	8500	1
4	13	xxxx	M	8000	1
5	12	fares	M	7000	2
6	9	sara	F	5800	2

نلاحظ انه لم يتم جلب أي استعلام أي انه لم يتم جلب أي data وانما يبقى في حالة الانتظار لانه يحاول الوصول وجلب بيانات ممسوحة من قبل session فينتظر ان تتحرر ال table وذلك عن طريق عمل ال commit/RollBack.

سؤال :

ماذا لو قمنا بإلغاء الاستعلام السابق وكتابة هذا الاستعلام

- SELECT * from dbo.emp
- Where empid=2;

هل سيقوم بجلب الداتا المطلوبة ام لا ؟

نعم، والسبب:

حين يقوم بتنفيذ الاستعلام يجد أن ال Lock القائم لا علاقة له بالاستعلام الذي ينفذ، وذلك لأن الجدول يملك pk. فالاستعلام ينفذ clustered seek، وأثناء التنفيذ يكتشف أن ال Dirty Data لا علاقة لها بال Row المطلوب فيعيده. بينما في الاستعلام السابق حين طلبنا منه جلب كامل ال Data وجد بعض الصفحات dirty ويتم الكتابة عليها لذلك لم يتم التنفيذ.

ولذلك يعد ال Index seek نوع من التحسين لأن احتمال مطابقة page يجري التعديل عليها dirty يصبح قليل.

لنقم الآن بتحرير الجدول وذلك عن طريق ال commit / rollback في ال session نفسها وسنقوم بعمل ال rollback وبعدها ل الاستعلام select نلاحظ انه القيمة xxxx لم تعد موجودة.

```
Select * from dbo.emp
Order by depid

BEGIN TRANSACTION
INSERT INTO dbo.emp VALUES ("xxxx",'m',8000,1)
ROLLBACK TRANSACTION
COMMIT TRANSACTION
```

الآن لنقم بمسك الجدول وذلك بتنفيذ التعليمة BEGIN TRANSACTION وذلك دون عمل ال COMMIT / ROLLBACK ولننتقل لقسم اخر (ال SESSION الثانية) ولنضيف التعليمة التالية قبل تعليمة ال SELECT معنى هذه التعليمة تجاهل القفل وقراءة ال DIRTY PAGE وهذا القرار بحال كانت البيانات هامة جدا

```
SELECT*
FROM dbo.emp
ORDER BY depid
BEGIN TRANSACTION
INSERT INTO dbo.emp VALUES('xxxx','m',8000,1);
```

ملاحظة هي بالسيشين الثاني
وهذه بالسيشين الأول:

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
SELECT * FROM dbo.emp
```

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
SELECT * FROM dbo.emp

163 %

	empid	ename	gender	salary	depid
1	1	ahmad	M	9400	1
2	2	sara	F	5800	2
3	3	maheer	m	80000	1
4	4	maha	f	8500	1
5	5	fares	M	7000	2
6	14	xxxx	m	8000	1

مع نتائج الاستعلام التي تظهر السجل الذي تم إدراجه ضمن ال Transaction الأولى ولم يتم عمل Commit له بعد

الآن سنقوم بعمل Transaction مع معالجة الـ Exception التي قد ترد معنا:
فنتطلب في حالة نجاح عمليتي الـ INSERT عمل COMMIT مع طباعة رسالة النجاح وفي حالة فشل إحداها عمل RollBack مع طباعة رسالة الخطأ الحاصل مع رقمها أو أي سطر وردت.
الان الاستعلامان ما بعده بالسيشين الثاني :

```
BEGIN TRY
    BEGIN TRANSACTION
        INSERT INTO dbo.emp VALUES ('aaaa', 'H', 8000, 1);
        INSERT INTO dbo.emp VALUES ('bbbb', 'F', 5000, 2);
    COMMIT TRANSACTION
    PRINT 'TRANSACTION COMMITED'
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION
    PRINT 'TRANSACTION ROLLED BACK'
    SELECT
        ERROR_NUMBER() AS ERRORNUMBER,
        ERROR_MESSAGE() AS ERRORMESSAGE,
        ERROR_LINE() AS ERRORLINE
END CATCH
```

شرح تفصيلي لكل جزء:

BEGIN TRY ... END TRY

- هذا الجزء يحتوي على الكود الذي نحاول تنفيذه.
- إذا لم يحدث أي خطأ، يتم تنفيذ كل الأوامر داخله بشكل طبيعي.

BEGIN TRANSACTION

- يبدأ معاملة (Transaction) جديدة.
- الهدف منها هو تنفيذ مجموعة من الأوامر كوحدة واحدة.
- إذا فشل أي أمر، يمكن التراجع عن كل الأوامر السابقة.

INSERT INTO dbo.emp VALUES(...)

- يتم إدخال صفين جديدين في جدول emp.
- القيم المدخلة تمثل: الاسم، الجنس، الراتب، ورقم القسم.

COMMIT TRANSACTION

- إذا نجحت كل أوامر الإدخال، يتم حفظ التغييرات نهائياً في قاعدة البيانات.

PRINT 'TRANSACTION COMMITED'

- طباعة رسالة تفيد بأن المعاملة تمت بنجاح.

BEGIN CATCH ... END CATCH

- إذا حدث خطأ في أي أمر داخل TRY، يتم الانتقال إلى هذا الجزء.

ROLLBACK TRANSACTION

- يتم التراجع عن كل التغييرات التي تمت داخل المعاملة.

PRINT 'TRANSACTION ROLLED BACK'

- طباعة رسالة تفيد بأنه تم التراجع عن المعاملة.

SELECT ERROR_NUMBER(), ERROR_MESSAGE(), ERROR_LINE()

- عرض معلومات عن الخطأ:
- ERROR_NUMBER(): رقم الخطأ.
- ERROR_MESSAGE(): وصف الخطأ.
- ERROR_LINE(): رقم السطر الذي حدث فيه الخطأ.

ماهو الهدف من الكود

- ضمان سلامة البيانات: إما أن يتم تنفيذ كل الأوامر بنجاح، أو لا يتم تنفيذ أي شيء.
 - معالجة الأخطاء: عرض معلومات مفيدة للمطور لتصحيح الخطأ.
 - تحسين الأمان: منع إدخال بيانات غير مكتملة أو خاطئة في حال حدوث مشكلة.
- وعند تنفيذ الاستعلام select في ال session الحالية او غيرها نلاحظ انه عمليتي الإضافة تم تنفيذيهما مع بعضهما البعض بنجاح وتمت الإضافة ال aaaa bbbbb

```

BEGIN TRY
    BEGIN TRANSACTION
    INSERT INTO dbo.emp VALUES ('aaaa', 'H',
    INSERT INTO dbo.emp VALUES ('bbbb', 'F',
    COMMIT TRANSACTION
    PRINT 'TRANSACTION COMMITED'
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION
    PRINT 'TRANSACTION ROLLED BACK'

```

163 %

Messages

(1 row affected)

(1 row affected)

TRANSACTION COMMITED

Completion time: 2025-12-24T20:31:38.5304923+03:00

Results		Messages				
empid	ename	gender	salary	depid		
1	ahmad	M	9400	1		
2	sara	F	5800	2		
3	maheer	m	80000	1		
4	maha	f	8500	1		
5	fares	M	7000	2		
6	xxxx	m	8000	1		
7	aaaa	H	8000	1		
8	bbbb	F	5000	2		
9	aaaa	H	8000	1		
10	bbbb	F	5000	2		

سنقوم ببناء TRANSACTION جديد، يشمل جزء منه وهو تعليمية إضافة بقيمة سالبة في الراتب، وقد سبق وأن تم وضع قيد على عمود الراتب في جدول الموظفين بشرط أن تكون القيمة المخزنة في خانة الراتب موجبة. (بالسيشين الثاني)

```
BEGIN TRY

BEGIN TRANSACTION

INSERT INTO dbo.emp (ename, gender, salary, depid)
VALUES ('AAAAA', 'F', 5000, 1);

PRINT 'TRANSACTION SUCCEEDED;'.

INSERT INTO dbo.emp (ename, gender, salary, depid)
VALUES ('BBBBB', 'M', 2000, 1);

PRINT 'TRANSACTION SUCCEEDED;'.

INSERT INTO dbo.emp (ename, gender, salary, depid)
VALUES ('CCCCC', 'M', -100, 1);

PRINT 'TRANSACTION SUCCEEDED;'.

COMMIT TRANSACTION;

END TRY

BEGIN CATCH

ROLLBACK TRANSACTION;

PRINT 'TRANSACTION ROLLED BACK;'.

SELECT

    ERROR_NUMBER() AS ErrorNumber,

    ERROR_MESSAGE() AS ErrorMessage,

    ERROR_LINE() AS ErrorLine;

END CATCH
```

لنقم الان بالتنفيذ ولننتقل الى ال messege ونلاحظ نجاح تنفيذ اول تعليمية وكذلك الثانية ومن ثم عمل ال RollBack بعد فشل الثالثة ومن ثم نجاح عملية الطباعة ل رسالة الخطا.

134 %

Results Messages

(1 row affected)
TRANSACTION SUCCEEDED.

(1 row affected)
TRANSACTION SUCCEEDED.

(0 rows affected)
TRANSACTION ROLLED BACK.

(1 row affected)

Completion time: 2025-12-24T20:50:06.0027761+03:00

ومحتوى رسالة الخطأ يعلمنا بأنه الخطأ
الحاصل بسبب القيد على عامود الراتب في
جدول الموظفين
لو قمنا بعمل استعلام الاتي :

134 %

Results Messages

	empid	ename	gender	salary	depid
1	1	ahmad	M	9400	1
2	3	maheer	m	80000	1
3	4	maha	f	8500	1
4	5	fares	M	7000	2
5	2	sara	F	5800	2

```
SELECT*
FROM dbo.emp
Order by depid
```

نلاحظ انه لم يتأثر أي row وذلك
بسبب ال rollback.

الخلاصة:

Transaction عنصر أساسي في قواعد البيانات، ويُستخدم لضمان تنفيذ العمليات الحساسة بطريقة آمنة تحافظ على تكامل البيانات، خاصة في الأنظمة المالية، الإدارية، والأنظمة التي تعتمد على أكثر من جدول مترابط.

أوامر Transaction الأساسية:

1. BEGIN TRANSACTION: بدء المعاملة.
2. COMMIT: تثبيت جميع التعديلات.
3. ROLLBACK: التراجع عن جميع التعديلات.
4. SAVEPOINT (اختياري): نقطة حفظ جزئية داخل Transaction.

الهدف من استخدام Transaction:

- الحفاظ على تكامل البيانات (Data Integrity).
- منع حدوث أخطاء ناتجة عن تنفيذ جزء من العمليات دون الجزء الآخر.
- ضمان أن البيانات إما أن تحفظ بالكامل أو لا تحفظ أبداً.

خصائص ACID الأربع لـ Transaction

تعتمد موثوقية وسلامة أي Transaction في قواعد البيانات على أربع خصائص أساسية تُعرف اختصاراً بـ ACID، وهي:

- **الذرية (Atomicity):** ويعني هذا المبدأ "الكل أو لا شيء". إما أن يتم تنفيذ جميع التعليمات ضمن الـ Transaction بالكامل بنجاح، أو لا يتم تنفيذ أي منها على الإطلاق، حيث يتم التراجع عن جميع التعديلات عبر أمر ROLLBACK.
- **الاتساق (Consistency):** تضمن هذه الخاصية أن الـ Transaction تنقل قاعدة البيانات من حالة متسقة (صحيحة) إلى حالة متسقة أخرى. يجب أن تلتزم العملية بكافة القواعد والقيود المعروفة مسبقاً في قاعدة البيانات (مثل قيود CHECK والمفاتيح الأجنبية FK).
- **العزل (Isolation):** تضمن هذه الخاصية أن العمليات المتزامنة لا تتداخل مع بعضها البعض. بمعنى، لا يمكن لـ Transaction أن ترى التغييرات الجزئية التي أجرتها Transaction أخرى لم يتم تثبيتها بعد (COMMIT). هذا يمنع مشكلة الـ Dirty Read ويرتبط مباشرة بآلية القفل (Locking) ومستويات العزل.
- **الديمومة (Durability):** تضمن أن التغييرات التي تم تثبيتها بنجاح عبر أمر COMMIT تكون دائمة. يتم تسجيل هذه التغييرات بشكل آمن على وسائط التخزين الثابتة (مثل القرص الصلب)، وتبقى محفوظة ومستمرة حتى في حالة فشل النظام أو انقطاع التيار الكهربائي.

مشكلة الـ Deadlock (الجمود) وأمثلتها

- تُعد مشكلة الـ Deadlock حالة متقدمة من التعارض تحدث نتيجة الوصول المتزامن للموارد المقفولة. تحدث مشكلة الـ Deadlock عندما تحتاج Transaction A إلى مورد (Resource) مقفول بواسطة Transaction B، وفي نفس الوقت، تحتاج Transaction B إلى مورد مقفول بواسطة Transaction A. هذا يخلق حلقة انتظار متبادل (Circular Wait)، حيث لا يمكن لأي من المعاملتين المتابعة، وتبقى كلتاها في حالة انتظار غير منتهية.
- على سبيل المثال، إذا بدأت Session 1 بقفل جدول Products ثم حاولت قفل جدول Suppliers، بينما بدأت Session 2 بقفل جدول Suppliers ثم حاولت قفل جدول Products، فستدخل العمليتان في Deadlock.
- يقوم نظام إدارة قواعد البيانات (DBMS) باكتشاف هذه الحالة تلقائياً، ويقوم باختيار إحدى المعاملات كضحية (Victim) لـ Deadlock (عادةً الأقل تكلفة)، ويُنفذ عليها أمر ROLLBACK (التراجع عن جميع تعديلاتها). هذا الإجراء يحرر القفل الذي كانت تحتجزه الضحية، مما يسمح للمعاملة الأخرى (الناجية) بالمتابعة وإتمام عملها بـ COMMIT.
- ولمعالجة الـ Deadlock والحد من احتمالية حدوثه، يجب تقليل نطاق القفل قدر الإمكان. يُعد استخدام الفهارس (Indexes) من أفضل الحلول حيث يساعد في تحويل عملية الوصول من Table Scan إلى Index Seek، مما يقلل نطاق القفل من قفل الجدول بالكامل إلى قفل

الصف (Row Lock) أو الصفحة (Page Lock) فقط. هذا يقلل من فرصة تداخل ال Transactions وبالتالي يقلل من احتمالية حدوث Deadlock.

🔗 سيناريو Deadlock:

- Session 1 (Transaction A) تبدأ المعاملة: ;BEGIN TRANSACTION
- Session 2 (Transaction B) تبدأ المعاملة في نفس الوقت: ;BEGIN TRANSACTION
- Session 1 تحجز قفلاً على جدول Products أولاً: UPDATE Products SET Price = 100 ;WHERE ID = 1
- Session 2 تحجز قفلاً على جدول Suppliers أولاً: UPDATE Suppliers SET Active = 0 ;WHERE SupplierID = 5
- Session 1 تحاول الآن الحصول على قفل على Suppliers (المقفول بواسطة Session 2): ;UPDATE Suppliers SET Note = 'Urgent' WHERE SupplierID = 5
- وفي نفس الوقت، Session 2 تحاول الحصول على قفل على Products (المقفول بواسطة Session 1): ;UPDATE Products SET Name = 'New' WHERE ID = 1
- **التفسير:** دخلت المعاملتان في حلقة انتظار متبادل (Deadlock). Transaction A تمتلك قفل Products وتحتاج قفل Suppliers، بينما Transaction B تمتلك قفل Suppliers وتحتاج قفل Products. النظام يكتشف هذه الحالة ويقوم بإلغاء إحدى المعاملات عبر ROLLBACK لتحرير الموارد والسماح للآخرى بالاستمرار.

